PI_Curs_06

6.0

Această lecție, **Capitolul 6: Detecția Obiectelor (Object Detection)**, marchează o tranziție de la sarcinile simple de clasificare a imaginilor la probleme de viziune computerizată mai complexe, care utilizează reprezentările interne avansate ale rețelelor neuronale.

Ghid de Studiu Complet: 6. Detecția Obiectelor (Object Detection)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 6 se concentrează pe aplicațiile complexe ale rețelelor neuronale convoluționale (CNN) în viziunea computerizată. Dacă în Capitolul 3 s-a studiat aplicarea diferitelor rețele CNN la sarcini simple de clasificare a imaginilor, acest capitol se bazează pe capacitatea rețelelor profunde de a reprezenta eficient imaginile pe **multiple niveluri**. Aceste reprezentări pe niveluri (*layer-wise representations*) s-au dovedit a fi fundamentale pentru succesul în sarcini mai dificile, cum ar fi **detecția obiectelor** și **segmentarea semantică**. La începutul capitolului sunt introduse și două metode cheie pentru îmbunătățirea generalizării modelului: **augmentarea imaginilor** și **fine-tuning-ul**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Fundamentul (Capitolul 3):** CNN-urile sunt folosite pentru sarcini simple de clasificare a imaginilor.
2. **Conceptul Cheie:** Rețelele profunde pot reprezenta eficient imaginile pe **multiple niveluri** (reprezentări pe niveluri/straturi).
3. **Aplicații Avansate:** Aceste reprezentări pe niveluri sunt esențiale pentru **detecția obiectelor** și **segmentarea semantică**.
4. **Tehnici de Generalizare:** La începutul capitolului sunt descrise **augmentarea imaginilor** și **fine-tuning-ul** ca metode de îmbunătățire a generalizării modelului.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6. Detecția Obiectelor

- **Fundamente (Revizuirea Capitolului 3):** În Capitolul 3 au fost studiate diverse rețele neuronale convoluționale utilizate frecvent în viziunea computerizată și aplicate sarcinilor

simple de clasificare a imaginilor.

- **Îmbunătățirea Generalizării:** La începutul acestui capitol vor fi descrise două metode care pot îmbunătăți generalizarea modelului:
  - **Augmentarea imaginilor (*Image augmentation*)**.
  - **Fine-tuning-ul**.
- **Reprezentarea Multi-Nivel:** Rețelele neuronale profunde pot reprezenta eficient imaginile pe multiple niveluri. Reprezentările pe niveluri (*layer-wise representations*) au fost utilizate cu succes în diverse sarcini de viziune computerizată.
- **Sarcini Complexe care Folosesc Reprezentări pe Niveluri:**
  - **Detecția obiectelor (*Object detection*)**.
  - **Segmentarea semantică (*Semantic segmentation*)**.
- **Obiectivul Capitolului 6:** Urmând ideea cheie de a valorifica reprezentările pe niveluri în viziunea computerizată, capitolul va începe cu componentele și tehnicile majore pentru detecția obiectelor.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Reprezentări pe Niveluri (*Layer-wise Representations*)

- **Explicație Simplificată (Scara de Observație):** O rețea neuronală profundă nu vede imaginea dintr-o singură perspectivă. Primele straturi învață *feature*-uri simple, cum ar fi marginile și colțurile. Straturile de mijloc combină aceste *feature*-uri simple în forme complexe (ex: ochi, roată). Straturile finale construiesc reprezentări complete (ex: fața unei persoane, o mașină). **Layer-wise representations** înseamnă că folosim informația din mai multe dintre aceste etape (straturi) pentru a rezolva o problemă, nu doar reprezentarea finală.
- *Asta se întâmplă deoarece:* Pentru sarcini complexe precum detecția obiectelor, avem nevoie de **feature-*uri de rezoluție înaltă** (din straturile inferioare) pentru a localiza cu precizie obiectul, dar și de** feature-*uri semantice** (din straturile superioare) pentru a ști ce este obiectul.

Concept Dificil: Object Detection vs. Semantic Segmentation

- **Detecția Obiectelor:** Implică **localizarea** (desenarea unei cutii de delimitare/bounding box) și **clasificarea** fiecărui obiect dintr-o imagine. Exemplu: "Aici este o mașină roșie (Bounding Box)".
- **Segmentarea Semantică:** Implică **clasificarea fiecărui pixel** dintr-o imagine, atribuind o etichetă de clasă întregii regiuni. Exemplu: "Toți acești pixeli aparțin clasei 'cer', toți acești pixeli aparțin clasei 'copac'".

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Evoluția de la Clasificare la Detecție):**
  - **CAUZĂ:** CNN-urile au învățat să producă reprezentări eficiente ale imaginilor în straturi.
  - **EFECT:** Această capacitate de reprezentare pe niveluri (care combină informații locale și globale) face posibilă abordarea sarcinilor care necesită localizare precisă și înțelegere semantică, cum ar fi **detecția obiectelor** și **segmentarea semantică**.
- **Implicația Tehnicilor de Generalizare:** Introducerea *image augmentation* și *fine-tuning* la începutul capitolului implică faptul că, deși CNN-urile sunt puternice, ele necesită adesea **strategii de transfer learning sau de creștere a datelor** pentru a avea succes în noile sarcini complexe de detecție.

---

5. EXERCITII (Conceptual și de Aplicare Teoretică)

Deoarece acest capitol este o introducere teoretică și nu conține formule de calcul, exercițiile se vor axa pe înțelegerea terminologiei și a relațiilor dintre capitole.

Exercițiul E.1: Identificarea Sarcinilor (Conceptual)

Identificați sarcinile complexe de viziune computerizată care utilizează cu succes reprezentările pe niveluri (*layer-wise representations*), conform sursei.

Rezolvare E.1:

1. **Detecția obiectelor (*Object detection*)**.
2. **Segmentarea semantică (*Semantic segmentation*)**.

Exercițiul E.2: Relația dintre Capitole (Analiză Teoretică)

De ce se face referire la Capitolul 3 și la studiul CNN-urilor în contextul noului capitol despre Detecția Obiectelor?

Rezolvare E.2:

Capitolul 3 a stabilit fundamentele prin studierea diverselor rețele neuronale convoluționale aplicate sarcinilor simple de clasificare. Aceste rețele CNN sunt **instrumentele de bază** utilizate pentru a genera reprezentările eficiente ale imaginilor pe multiple niveluri (layer-wise representations), care sunt la rândul lor cheia pentru a aborda sarcinile complexe de detecție și segmentare.

Exercițiul E.3: Metode de Îmbunătățire a Generalizării (Clasificare)

Numiți și clasificați cele două metode care vor fi descrise la începutul Capitolului 6, al căror scop este îmbunătățirea generalizării modelului.

Rezolvare E.3:

1. **Augmentarea imaginilor (*Image augmentation*)**.
2. **Fine-tuning-ul**.

6.1

Suntem la începutul Capitolului 6, iar prima lecție, **6.1 Image augmentation (Augmentarea Imaginilor)**, este esențială deoarece abordează o problemă fundamentală în *deep learning* pentru viziunea computerizată: **nevoia de seturi de date mari** și îmbunătățirea capacității de generalizare a modelelor.

Ghid de Studiu Complet: 6.1 Augmentarea Imaginilor (Image augmentation)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 6.1 tratează **augmentarea imaginilor (*Image augmentation*)** ca o tehnică vitală pentru succesul rețelelor neuronale profunde. Această metodă generează **exemple de antrenament similare, dar distincte**, aplicând o serie de modificări aleatorii imaginilor originale, extinzând astfel efectiv dimensiunea setului de date. Prin introducerea acestei varietăți, modelul învață să se bazeze mai puțin pe atribute specifice și irelevante (cum ar fi poziția exactă a obiectului sau condițiile de iluminare), îmbunătățind semnificativ **capacitatea de generalizare**. Succesul acestei tehnici a fost probabil indispensabil pentru modele timpurii precum AlexNet.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul Principal:** Extinderea dimensiunii setului de date prin generarea de exemple de antrenament distincte din imagini similare.
2. **Beneficiu Major:** Îmbunătățirea generalizării modelului prin reducerea dependenței de atribute specifice (ex: poziția obiectului sau culoarea).
3. **Metode Geometrice:** Răsturnarea pe orizontală (*flipping left and right*) și decuparea aleatorie (*randomly crop*).
4. **Metode Cromatice:** Ajustarea factorilor precum luminozitatea (*brightness*), contrastul, saturația și nuanța (*hue*).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.1 Augmentarea Imaginilor

Principiul și Motivația

- Seturile de date mari sunt o condiție prealabilă pentru succesul rețelelor neuronale profunde în diverse aplicații.
- Augmentarea imaginilor generează exemple de antrenament distincte prin aplicarea unei serii de **schimbări aleatorii** imaginilor.
- Această tehnică îmbunătățește generalizarea deoarece permite modelelor să se bazeze mai puțin pe atribute specifice.
- Majoritatea metodelor de augmentare a imaginilor au un anumit grad de **aleatorism**.

Tipuri de Augmentări

1. **Răsturnarea (Flipping)**
   - **Orizontală (*Left and Right*):** Aceasta este una dintre cele mai timpurii și mai utilizate metode. De obicei, răsturnarea stânga-dreapta nu schimbă categoria obiectului. Sursa menționează aplicarea răsturnării stânga-dreapta cu o șansă de **50%**.
   - **Verticală (*Up and Down*):** Răsturnarea sus-jos nu este la fel de comună ca cea stânga-dreapta, dar se aplică tot cu o șansă de **50%**.
2. **Decuparea (*Cropping*) și Scalarea**
   - Putem decupa o imagine în moduri diferite pentru a face ca obiectul de interes să apară în poziții diferite.
   - Decuparea aleatorie a imaginii reduce dependența unui model de **poziția obiectului**.
   - Decuparea poate face ca obiectele să apară la **scări diferite** și poate reduce sensibilitatea modelului la poziția țintă.
   - Un exemplu detaliat de decupare: se decupează aleatoriu o zonă cu o suprafață între **10% și 100%** din aria originală.
   - Raportul lățime/înălțime al zonei decupate este selectat aleatoriu, de obicei între **0.5 și 2**.
   - Dimensiunea (lățimea și înălțimea) regiunii decupate este apoi **scalată la 200 de pixeli**.
3. **Modificarea Culorilor (*Changing Colors*)**
   - Putem ajusta factori precum **luminozitatea (*brightness*), contrastul, saturația și nuanța (*hue*)** pentru a reduce sensibilitatea modelului la culoare.
   - **Luminozitatea:** Poate fi schimbată aleatoriu la o valoare între **50% (1 – 0.5)** și **150% (1 + 0.5)** din imaginea originală.
   - Se pot schimba aleatoriu toți cei patru aspecte (luminozitatea, contrastul, saturația și nuanța) în același timp.

Practica (Combinarea Metodelor)

- În practică, se pot combina **metode multiple** de augmentare a imaginilor și se aplică fiecărei imagini.
- Pentru a obține rezultate definitive în timpul predicției, se aplică augmentarea imaginilor **doar la exemplele de antrenament** și **nu** se folosește augmentarea cu operații aleatorii în timpul predicției.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Generalizare vs. Memorare

- **Analogie (Învățarea Limbii):** Dacă un copil învață cuvântul "mașină" văzând doar mașini roșii parcate într-o anumită poziție, el ar putea memora (overfit) că "mașină" înseamnă un obiect roșu în acel loc. Dacă însă i se arată mașini de culori diferite (prin ajustarea luminozității/culorii) și în locații diferite ale câmpului vizual (prin decupare/răsturnare), copilul învață conceptul general de "mașină", **îmbunătățindu-și generalizarea**.

Concept Dificil: Rolul Aleatorismului în Augmentare

- *Asta se întâmplă deoarece:* Deoarece cele mai multe metode de augmentare includ un grad de aleatorism, aplicarea repetată a augmentării pe aceeași imagine va produce o serie de variații distincte. Dacă operațiile nu ar fi aleatorii (ex: am răsturna mereu imaginea), modelul ar vedea doar o singură variație nouă, limitând beneficiile extinderii setului de date.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Poziție vs. Generalizare):**
  - **CAUZĂ:** Fără augmentare, modelul s-ar putea baza pe poziția obiectului (de exemplu, pisica este mereu în centrul imaginii).
  - **EFECT: Decuparea aleatorie** forțează modelul să ignore poziția absolută și să se concentreze pe *feature*-urile intrinseci ale obiectului, reducând dependența de poziția țintă și îmbunătățind generalizarea.
- **Conexiunea cu Capitolul 3.5 (Pooling):** Sursa menționează că *pooling layer* poate reduce sensibilitatea unui strat convoluțional la poziția țintă. Augmentarea prin decupare completează acest mecanism, reducând în plus sensibilitatea la poziția și scala obiectului.

---

5. EXERCITII (Teorie și Calcul Aplicat)

Deoarece sursa conține instrucțiuni precise despre intervale de probabilitate și scalare, putem genera exerciții de aplicare numerică.

Exercițiul A.4: Calcul Aplicat (Augmentarea Luminozității)

O imagine are o valoare medie a pixelilor $V_{oriina} = 150$. Se aplică o augmentare a luminozității (brightness) conform specificațiilor sursei, unde luminozitatea este schimbată aleatoriu între **50% și 150%** a valorii originale.

1. Determinați intervalul numeric al factorului de scalare (intervalul $[a,]$ din sursă).
2. Calculați valoarea minimă și maximă (V_{min}, V_{max}) pe care o poate avea valoarea medie a pixelilor după această augmentare.

Rezolvare A.4:

1. **Intervalul Factorului de Scalare ():** Sursa indică schimbarea între 50 și 150.

$$_{min} = 1 - 0.5 = \mathbf{0.5}$$
$$_{max} = 1 + 0.5 = \mathbf{1.5}$$

   Intervalul factorului de scalare este $[\mathbf{0.5}, \mathbf{1.5}]$.
2. **Valoarea Minima și Maxima a Pixelilor:**

$$V_{min} = V_{oriina} \times _{min} = 150 \times 0.5 = \mathbf{75}$$
$$V_{max} = V_{oriina} \times _{max} = 150 \times 1.5 = \mathbf{225}$$

Exercițiul A.5: Analiză Dimensională și Procentuală (Decupare Aleatorie)

O imagine de intrare are dimensiunea de 400×500 de pixeli.

1. Care este aria maximă (în pixeli pătrați) a unei regiuni decupate, conform intervalului specificat în sursă (10% - 100% din aria originală)?
2. Dacă o regiune decupată este scalată la 200 de pixeli, ce implică acest lucru pentru modelul de *deep learning* care urmează să proceseze imaginea?

Rezolvare A.5:

1. **Aria Maximă a Regiunii Decupate:**
   - Aria originală $A_{ori} = 400 \times 500 = 200,000$ pixeli pătrați.
   - Aria maximă decupată este 100 din aria originală.

$$A_{max} = 1.00 \times 200,000 = \mathbf{200,000} \text{ pixeli pătrați}$$

2. **Implicația Scalării la 200 de Pixeli:** Scalarea lățimii și înălțimii regiunii decupate la 200 de pixeli înseamnă că **toate** imaginile de antrenament, indiferent de dimensiunea inițială

sau de mărimea și scara obiectului decupat, vor fi introduse în rețeaua CNN cu o dimensiune standardizată, de obicei 200×200 (dacă este scalată la aceeași înălțime și lățime). Acest lucru este necesar pentru ca rețelele neuronale convoluționale să poată procesa *batch*-uri de imagini cu aceeași dimensiune.

Exercițiul A.6: Teorie (Predictie vs. Antrenament)

Conform sursei, de ce se aplică augmentarea imaginilor **doar** la setul de antrenament și nu se folosește în timpul predicției?

Rezolvare A.6:

Augmentarea imaginilor, în special cea cu operații aleatorii, este concepută pentru a crea **varietate** și a împiedica modelul să *overfit*-eze (să memoreze) setul de date. Totuși, în timpul predicției, obiectivul este de a obține **rezultate definitive și stabile** pentru o singură imagine de intrare. Aplicarea operațiilor aleatorii (cum ar fi răsturnarea sau modificarea aleatorie a luminozității) în timpul predicției ar genera rezultate inconsistente de fiecare dată când aceeași imagine ar fi introdusă, făcând predicția inutilă.

6.2

Lecția **6.2 Fine-tuning** este crucială, deoarece prezintă o metodă standard și extrem de eficientă, cunoscută sub numele de *transfer learning*, pentru a antrena modele puternice de *deep learning* atunci când resursele de date sau costurile sunt limitate.

Ghid de Studiu Complet: 6.2 Fine-tuning

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Fine-tuning-ul este o tehnică comună de **transfer learning** menită să rezolve problema costului și timpului necesar pentru a colecta seturi de date mari și etichetate (cum ar fi ImageNet, a cărui colectare a costat milioane de dolari). Deoarece dimensiunea seturilor de date întâlnite în practică se situează adesea între seturi mici (precum Fashion-MNIST, 60.000 de imagini) și seturi masive (precum ImageNet, peste 10 milioane de imagini), fine-tuning-ul permite transferul de cunoștințe dintr-un model antrenat pe un set mare (*source dataset*) la un model nou, adaptat pentru un set mai mic (*target dataset*). Modelul pre-antrenat extrage *feature*-uri de imagine generale (margini, texturi, forme), iar fine-tuning-ul ajustează aceste *feature*-uri pentru sarcina specifică, îmbunătățind semnificativ **capacitatea de generalizare** a modelului, în special când setul țintă este mult mai mic.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Contextul Problemă:** Seturile de date necesare (ex: ImageNet) sunt extrem de costisitoare și consumatoare de timp pentru colectare și etichetare.

2. **Soluția:** Aplicarea **transfer learning** prin *fine-tuning*.
3. **Transferul de Cunoștințe:** Modelul pre-antrenat extrage cunoștințe generale despre imagini (muchii, texturi, forme) care sunt utile pentru sarcina țintă.
4. **Mecanismul Cheie:** Se copiază designul și toți parametrii modelului sursă, **cu excepția stratului de ieșire**, care este înlocuit și antrenat *from scratch*, în timp ce restul parametrilor sunt ajustați (fine-tuned).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.2 Fine-tuning

Contextul Seturilor de Date

- S-au discutat seturi de date precum Fashion-MNIST, care are doar **60.000 de imagini**.
- ImageNet este cel mai utilizat set de date la scară largă în mediul academic, având **mai mult de 10 milioane de imagini** și **1000 de obiecte**.
- Dimensiunea seturilor de date întâlnite în mod obișnuit se situează între aceste două extreme.
- O soluție evidentă la lipsa datelor este colectarea de date suplimentare, dar acest lucru necesită mult timp și bani (cazul ImageNet a costat milioane de dolari din finanțări de cercetare).

Transfer Learning și Fine-tuning

- O altă soluție este aplicarea **transfer learning** pentru a transfera cunoștințele învățate de la *source dataset* la *target dataset*.
- Cunoștințele transferate: Chiar dacă imaginile din ImageNet nu au legătură directă cu sarcina țintă (ex: scaune), modelul pre-antrenat extrage _*feature*-uri generale_* (margini, texturi, forme, compoziția obiectului) care sunt eficiente pentru recunoașterea obiectelor țintă.
- Fine-tuning-ul este o tehnică comună în *transfer learning*.

Cei Patru Pași ai Fine-tuning-ului (Figura 1)

1. **Pre-antrenare:** Se pre-antrenează un model de rețea neuronală (*source model*) pe un *source dataset* (de exemplu, setul de date ImageNet).
2. **Crearea Modelului Țintă:** Se creează un nou model de rețea neuronală (*target model*). Acesta copiază designul și toți parametrii modelului sursă, **cu excepția stratului de ieșire (*output layer*)**.
3. **Adăugarea Stratului de Ieșire:** Se adaugă un nou strat de ieșire la modelul țintă. Numărul de ieșiri al acestui strat este egal cu numărul de categorii din setul de date țintă. Parametrii acestui nou strat sunt **inițializați aleatoriu**.

4. **Antrenarea Finală (Fine-tuning):** Se antrenează modelul țintă pe *target dataset* (de exemplu, un set de date cu scaune). Stratul de ieșire nou adăugat este antrenat **de la zero (*from scratch*)**, în timp ce parametrii tuturor celorlalte straturi sunt **fine-tuned** (ajustați fin) pe baza parametrilor modelului sursă.

Rezultatul: Fine-tuning-ul ajută la îmbunătățirea capacității de generalizare a modelelor, mai ales când seturile de date țintă sunt mult mai mici decât seturile de date sursă.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Transfer Learning

- **Analogie Simplificată (A învăța să desenezi):** Transfer learning este ca și cum ai folosi cunoștințele dobândite învățând să desenezi portrete (sarcina sursă) pentru a învăța apoi să desenezi peisaje (sarcina țintă). Deși portretele și peisajele sunt diferite, abilitățile de bază (cum să ții creionul, să redai umbrele, să înțelegi perspectivele) sunt transferabile. Fine-tuning-ul ajustează apoi aceste abilități de bază pentru a desena eficient elemente specifice peisajului (copaci, nori, munți).

Concept Dificil: De ce excludem Stratul de Ieșire (Output Layer)?

- *Asta se întâmplă deoarece:* Stratul de ieșire al modelului sursă este extrem de specific și dependent de etichetele setului de date sursă (de exemplu, 1000 de clase ImageNet).
- Dacă setul de date țintă are un număr diferit de categorii (de exemplu, 5 categorii de scaune), stratul de ieșire original este **incompatibil dimensional** și **irelevant semantic**. Înlocuirea și inițializarea sa aleatorie permite modelului să învețe *mapping*-ul specific de la *feature*-urile generale (Layer 1 la L-1) la noile etichete țintă.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Cost vs. Performanță):** Costul ridicat și timpul necesar pentru colectarea datelor etichetate (cauză) fac antrenarea de la zero (*training from scratch*) nepractică pentru majoritatea aplicațiilor. Fine-tuning-ul (efect) oferă o cale rapidă și eficientă de a obține performanțe ridicate chiar și pe seturi de date limitate, prin exploatarea cunoștințelor preexistente.
- **Conexiunea cu Generalizarea:** Când setul de date țintă este mic, antrenarea întregului model de la zero ar duce rapid la *overfitting* (memorare). Prin păstrarea ponderilor pre-antrenate pentru majoritatea straturilor (Layer 1 la L-1), se impune o formă de

regularizare, forțând modelul să se bazeze pe *feature*-urile generale și robuste învățate pe setul mare, îmbunătățind generalizarea.

---

5. EXERCITII (Teorie și Analiză Conceptuală)

Deoarece sursa se axează pe pași procedurali și pe contextul de utilizare, exercițiile vor fi axate pe înțelegerea procesului și a implicațiilor dimensionale/financiare.

Exercițiul F.1: Aplicarea Pașilor Fine-tuning (Ordine și Acțiune)

Presupuneți că doriți să folosiți un model pre-antrenat pe ImageNet (1000 de clase) pentru a clasifica 20 de categorii noi de flori. Descrieți acțiunea exactă întreprinsă asupra parametrilor stratului de ieșire și a parametrilor straturilor ascunse (Layer 1 la L-1) în timpul antrenamentului pe setul de date cu flori.

Rezolvare F.1:

1. **Stratul de Ieșire Original (Output Layer):** Acesta este **eliminat** (datorită incompatibilității dimensionale/semantice) și **înlocuit** cu un nou strat care are 20 de noduri de ieșire (corespunzând celor 20 de clase de flori). Parametrii acestui nou strat sunt **inițializați aleatoriu** și **antrenați *from scratch***.
2. **Straturile Ascunse (Layer 1 la L-1):** Parametrii acestor straturi sunt **copiați** de la modelul sursă și apoi sunt **fine-tuned** (ajustați fin) pe setul de date țintă cu o rată de învățare mică, păstrând astfel cunoștințele generale acumulate.

Exercițiul F.2: Analiza Datelor și Costului (Conceptual)

Comparați setul de date Fashion-MNIST și ImageNet în termeni de dimensiune și comentați implicația costului mare de colectare a datelor.

Rezolvare F.2:

- **Fashion-MNIST:** Conține **60.000 de imagini**.
- **ImageNet:** Conține **mai mult de 10 milioane de imagini** și 1000 de obiecte.
- **Implicația Costului:** Colectarea și etichetarea unui set de date la scara ImageNet necesită un efort financiar și temporal imens, ajungând la **milioane de dolari**. Acest cost face ca antrenarea de la zero pe seturi de date mari să nu fie o opțiune realistă pentru majoritatea cercetătorilor sau companiilor, justificând astfel utilizarea *fine-tuning*-ului, care revalorifică efortul inițial.

6.3

Absolut! Lecția **6.3 Detecția Obiectelor și Cutiile de Delimitare (Object Detection and Bounding Boxes)** face trecerea esențială de la clasificarea simplă a imaginii la sarcini complexe care necesită atât recunoașterea, cât și localizarea spațială.

Ghid de Studiu Complet: 6.3 Detecția Obiectelor și Cutiile de Delimitare

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 6.3 definește detecția obiectelor (*object detection* sau *object recognition*) ca fiind sarcina de a identifica **multiple obiecte** dintr-o imagine și de a determina **pozițiile lor specifice**. Aceasta diferă de sarcinile anterioare de clasificare a imaginilor (discutate în Secțiunile 3.7–3.10), unde se presupunea că există un singur obiect major în imagine. Detecția obiectelor este crucială și aplicată pe scară largă în domenii practice, cum ar fi mașinile autonome, unde este necesară detectarea pozițiilor vehiculelor, pietonilor și obstacolelor, sau în sistemele de securitate. Mecanismul primar utilizat pentru a descrie localizarea spațială a unui obiect este **cutia de delimitare (*bounding box*)**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Diferența Față de Clasificare:** Detecția obiectelor implică recunoașterea **mai multor obiecte** și cunoașterea **poziției** lor, nu doar a categoriei.
2. **Instrumentul de Localizare: Cutia de delimitare (*bounding box*)** este utilizată pentru a descrie localizarea spațială a unui obiect.
3. **Reprezentarea Cutiilor:** O cutie de delimitare dreptunghiulară poate fi determinată fie prin coordonatele x și y ale colțului din stânga sus și ale colțului din dreapta jos, fie prin coordonatele centrului (x, y), lățimea și înălțimea cutiei.
4. **Sistemul de Coordonate:** Originea $(0, 0)$ a coordonatelor în imagine este **colțul din stânga sus** al imaginii. Direcțiile pozitive sunt spre dreapta (axa x) și în jos (axa y).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.3 Detecția Obiectelor și Cutiile de Delimitare

Definirea Sarcinii

- În sarcinile anterioare de clasificare a imaginilor (Secțiunile 3.7–3.10), se presupunea un singur obiect major.
- În realitate, imaginile conțin adesea **multiple obiecte** de interes.
- Detecția obiectelor (sau recunoașterea obiectelor) necesită cunoașterea **categoriilor** obiectelor și a **pozițiilor lor specifice** în imagine.

Aplicații (Exemple practice)

- Detecția obiectelor este utilizată pe scară largă.

- **Mașini autonome (*Self-driving*)**: Necesitatea de a detecta pozițiile vehiculelor, pietonilor, drumurilor și obstacolelor în imaginile video capturate.
- **Robotică**: Roboții folosesc această tehnică pentru a detecta și localiza obiecte de interes în mediul lor.
- **Sisteme de Securitate**: Necesitatea de a detecta obiecte anormale, cum ar fi intrușii sau bombele.

Localizarea Spațială (Bounding Boxes)

- În detecția obiectelor, se folosește de obicei o **cutie de delimitare (*bounding box*)** pentru a descrie localizarea spațială a unui obiect.
- Cutia de delimitare este **dreptunghiulară**.

Modalități de Reprezentare a Cutiilor de Delimitare

Există două reprezentări comune:

1. **Colțuri:** Determinată de coordonatele x și y ale:
   - Colțului din **stânga sus** al dreptunghiului.
   - Colțului din **dreapta jos** al dreptunghiului.
2. **Centru și Dimensiuni:** Determinată de:
   - Coordonatele (x, y) ale **centrului** cutiei.
   - **Lățimea (*width*)** și **înălțimea (*height*)** cutiei.

Sistemul de Coordonate al Imaginii

- Originea coordonatelor $(0, 0)$ este situată în **colțul din stânga sus** al imaginii.
- Direcția pozitivă a axei x este **spre dreapta**.
- Direcția pozitivă a axei y este **în jos**.

Exemplu de Imagine:

- O imagine mostră este prezentată cu un câine (pe partea stângă) și o pisică (pe partea dreaptă).
- Cutii de delimitare sunt desenate pentru a cuprinde contururile principale ale celor două obiecte.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Bounding Box (Cutia de Delimitare)

- **Analogie Simplificată (Fotografierea):** Dacă clasificarea este ca și cum ai spune "Aceasta este o pisică", detecția obiectelor este ca și cum ai spune "Aceasta este o

pisică, iar ea se află într-o ramă (bounding box) care începe aici și se termină acolo." Bounding box-ul este rama minimă dreptunghiulară care înconjoară complet obiectul de interes.

Concept Dificil: Sistemul de Coordonate Y Pozitiv în Jos

- *Asta se întâmplă deoarece:* În grafica digitală și viziunea computerizată, este o convenție standard ca axa y să crească în direcția descendentă. Astfel, $(0, 0)$ este colțul din stânga sus. Dacă o imagine are o înălțime de 500 de pixeli, rândul de jos al imaginii se află la $y = 500$.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Complexitatea Sarcinii):**
  - **CAUZĂ:** Sarcinile de viziune din lumea reală implică obiecte multiple și necesită cunoașterea locației.
  - **EFECT:** Detecția obiectelor devine o sarcină cu două sub-obiective: 1) **Clasificare** (Ce este?) și 2) **Regresie** (Unde este?). Antrenarea unui model de detecție implică, prin urmare, prezicerea atât a etichetei clasei, cât și a celor patru valori numerice care definesc cutia de delimitare (ex: $x_{min}, y_{min}, x_{max}, y_{max}$).

---

5. EXERCITII (Calcul și Analiză bazate pe Coordonate)

Aceste exerciții testează înțelegerea sistemului de coordonate și a celor două reprezentări ale cutiei de delimitare.

Exercițiul B.1: Conversia Reprezentării Cutiei de Delimitare (Colțuri la Centru)

Se consideră o cutie de delimitare pentru un obiect, definită de coordonatele colțurilor sale (stânga sus și dreapta jos).

- **Colțul Stânga Sus:** $(x_{min}, y_{min}) = (100, 50)$
- **Colțul Dreapta Jos:** $(x_{max}, y_{max}) = (300, 250)$

Determinați reprezentarea cutiei de delimitare folosind **coordonatele centrului** (x_c, y_c)**, lățimea (**w**) și înălțimea (**h**)**.

Rezolvare B.1:

1. **Lățimea (**w**) și Înălțimea (**h**):**

$$w = x_{max} - x_{min} = 300 - 100 = \mathbf{200}$$

$$h = y_{max} - y_{min} = 250 - 50 = \mathbf{200}$$

2. **Coordonatele Centrului (x_c, y_c):**

$$x_c = x_{min} + w/2 = 100 + 200/2 = 100 + 100 = \mathbf{200}$$

$$y_c = y_{min} + h/2 = 50 + 200/2 = 50 + 100 = \mathbf{150}$$

Reprezentarea centrului și dimensiunilor este: $(x_c, y_c, w, h) = (200, 150, 200, 200)$.

Exercițiul B.2: Analiza Poziției Obiectelor (Vizual și Coordonate)

Referindu-ne la imaginea mostră din sursă (Câine la stânga, Pisică la dreapta):

1. Pentru Câine (Bounding Box Albastru): Estimați o pereche de coordonate (x, y) pentru centrul cutiei, ținând cont de axele numerotate (0-600 pentru x, 0-500 pentru y).
2. De ce, în acest sistem de coordonate, o valoare $y = 500$ este mai jos pe imagine decât $y = 100$?

Rezolvare B.2:

1. **Estimarea Centru Câine (Bounding Box Albastru):**
   - Câinele este în stânga. Cutia albastră pare să se întindă pe x de la aproximativ 50 la 350, și pe y de la aproximativ 50 la 500.
   - $x_c \approx (50 + 350)/2 = \mathbf{200}$
   - $y_c \approx (50 + 500)/2 = \mathbf{275}$
   - (O estimare rezonabilă ar fi $(200, 275)$).
2. **Explicația Ordinii Y:**
   - În viziunea computerizată (conform sursei), originea $(0, 0)$ este colțul din stânga sus al imaginii.
   - Axa y pozitivă este orientată **în jos**.
   - Prin urmare, o valoare $y = 500$ reprezintă o poziție mai aproape de marginea de jos a imaginii decât $y = 100$, care este mai aproape de marginea de sus.

6.4.

Lecția **6.4 Anchor boxes (Cutiile Ancoră)** introduce un concept fundamental care stă la baza multor algoritmi moderni de detecție a obiectelor. Mecanismul abordează problema complexă a găsirii și localizării obiectelor de diferite dimensiuni și forme într-o singură trecere a rețelei.

Ghid de Studiu Complet: 6.4 Anchor boxes

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Algoritmii de detecție a obiectelor abordează sarcina prin eșantionarea unui număr mare de regiuni în imaginea de intrare. **Cutiile Ancoră (*Anchor boxes*)** reprezintă o metodă de eșantionare care generează cutii de delimitare multiple cu scări (s) și rapoarte de aspect (r) variabile, centrate pe **fiecare pixel** al imaginii. Modelul de detecție învață apoi, pentru fiecare cutie ancoră, dacă aceasta conține un obiect de interes și care este **decalajul (offset)** necesar pentru a ajusta cutia ancoră la cutia de delimitare reală (*ground-truth*). Similaritatea dintre cutia ancoră și cutia *ground-truth* este măsurată prin **Intersecția peste Uniune (IoU)**. În final, **Suprimarea Non-Maximă (NMS)** este utilizată pentru a simplifica ieșirea, eliminând cutiile redundante.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Rolul Anchor Boxes:** Generarea unui set mare de regiuni propuse, variind scara și raportul de aspect, centrate pe fiecare pixel.
2. **Etichetarea în Antrenament:** Fiecare cutie ancoră primește o etichetă de **clasă** și o etichetă de **offset** (regresie).
3. **Măsura de Similaritate: IoU** (Intersection over Union), bazată pe indicele Jaccard, măsoară suprapunerea între cutiile ancoră și cutiile *ground-truth*.
4. **Transformarea Offset-ului:** Se aplică o transformare neliniară (logaritmică și scalată) asupra poziției și dimensiunii relative pentru a obține *offset*-uri distribuite uniform, mai ușor de antrenat.
5. **Post-Procesare: Non-Maximum Suppression (NMS)** elimină predicțiile multiple și suprapuse pentru același obiect.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

Generarea Cutiilor Ancoră

- **Scop:** Algoritmii eșantionează un număr mare de regiuni, determină dacă ele conțin obiecte de interes, și ajustează limitele regiunilor pentru a prezice cutiile *ground-truth* mai precis.
- **Design:** Cutiile Ancoră ($anchor$ boxes) sunt cutii de delimitare multiple, generate cu scări (s) și rapoarte de aspect (r) variabile, centrate pe **fiecare pixel** al imaginii.
- **Definiții:**
  - h și w: înălțimea și lățimea imaginii de intrare.
  - **Scala (s):** $s \in$.
  - **Raportul de Aspect (r):** Raportul lățime/înălțime, $r > 0$.
- **Formulele Dimensiunilor:**
  - Lățime cutie ancoră: $ws\sqrt{r}$.
  - Înălțime cutie ancoră: $hs/\sqrt{r}$.

- **Complexitatea Totală:** Dacă se folosesc n scări ($s_1, \ldots, s_n$) și m rapoarte de aspect ($r_1, \ldots, r_m$), iar cutiile sunt centrate pe toți cei $w \times h$ pixeli, numărul total de cutii ancore este $whnm$.
- **Simplificare Practică:** Deși $whnm$ poate fi prea mare, în practică se consideră doar combinațiile care conțin s_1 sau r_1.
  - Numărul de cutii ancoră centrate pe același pixel este $n + m - 1$.
  - Numărul total de cutii ancoră generate este $wh(n + m - 1)$.

Măsurarea Similarității: Intersecția peste Uniune (IoU)

- Similaritatea dintre o cutie ancoră și o cutie *ground-truth* este măsurată prin **Indexul Jaccard**.
- În contextul cutiilor de delimitare, acesta este numit **Intersection over Union (IoU)**, care este raportul dintre aria intersecției și aria uniunii seturilor de pixeli ale celor două cutii.
- **Formula IoU:** $(A, B) = \frac{AB}{AB}$.
- **Intervalul IoU:** IoU este cuprins între 0 și 1.
  - $o = 0$: Cutii care nu se suprapun deloc.
  - $o = 1$: Cele două cutii sunt identice.

Etichetarea Cutiilor Ancoră pentru Antrenament

- Fiecare cutie ancoră este tratată ca un exemplu de antrenament.
- Pentru a antrena modelul de detecție, sunt necesare etichete de **clasă** și **offset** pentru fiecare cutie ancoră.
- **Clasa:** Eticheta de clasă este clasa obiectului relevant (ex: câine, pisică).
- **Offset:** Eticheta *offset* este decalajul cutiei *ground-truth* față de cutia ancoră.

Algoritmul de Alocare (Assignment)

- Scop: Alocarea cutiilor *ground-truth* (B_j) la cele mai apropiate cutii ancoră (A_i).
- Se definește o matrice $\mathbf{X} \in \mathbb{R}^{n_a \times n}$, unde x_{ij} este IoU între A_i și B_j ($n_a \quad n$).
- **Pașii Alocării (Iterativ):**
  1. Se găsește elementul maxim din $\mathbf{X}$ (ex: $x_{i_1 j_1}$). Cutia B_{j_1} este alocată cutiei A_{i_1}.
  2. Se elimină rândul i_1 și coloana j_1 din $\mathbf{X}$.
  3. Se repetă pașii 1 și 2 până când toate coloanele (toate cutiile *ground-truth*) sunt epuizate. Astfel, fiecare cutie *ground-truth* este alocată la cel puțin o cutie ancoră (cea mai apropiată).
- **Etichetarea Cutiilor Rămase:** Cutiile ancoră rămase sunt etichetate cu B_j dacă IoU depășește un prag predefinit (ex: 0.5).
- **Cutiile Negative:** Dacă o cutie ancoră nu este alocată niciunei cutii *ground-truth*, clasa sa este etichetată ca "background" (cutii negative).

Transformarea Offset-ului (Etichetarea Decalajului)

Pentru a obține *offset*-uri mai ușor de ajustat, se folosește o transformare bazată pe coordonatele centrale și dimensiunile cutiilor ancoră (A) și *ground-truth* (B).

- **Formula de Offset Etichetat:**

$$\frac{x - x_a}{{}_x w_a} - {}_x, \frac{y - y_a}{{}_y h_a} - {}_y, \frac{\text{lo}\,\frac{w}{w_a} - {}_w}{{}_w}, \frac{\text{lo}\,\frac{h}{h_a} - {}_h}{{}_h}$$

- **Constante Implicite:** ${}_x = {}_y = {}_w = {}_h = 0$, ${}_x = {}_y = 0.1$, ${}_w = {}_h = 0.2$.
- Offset-urile claselor negative (background) nu ar trebui să afecteze funcția obiectiv.

Suprimarea Non-Maximă (Non-maximum suppression - NMS)

- **Nevoia:** În timpul predicției, se generează multe cutii ancoră prezise care se suprapun semnificativ, toate încercând să încadreze același obiect.
- **Scopul NMS:** Simplificarea ieșirii prin fuzionarea cutiilor similare prezise care aparțin aceluiași obiect.
- **Procedura:**
  1. Cutiile prezise non-background sunt sortate într-o listă în ordine descrescătoare a scorului de **încredere (*confidence*)**.
  2. Se selectează cutia B_1 (cea mai mare încredere) ca bază și se elimină din toate cutiile care au un $o(B_1, B_i)$ care depășește un prag .
  3. Se repetă procesul cu următoarea cutie cu cea mai mare încredere rămasă (B_2), eliminând cutiile suprapuse, până când toți candidații au fost utilizați ca bază.
  4. Se afișează cutiile rămase în .

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Ground-truth Bounding Box

- **Explicație Simplificată (Adevărul Fundamental):** Aceasta este cutia de delimitare **reală**, eticheta corectă, marcată manual de un om în setul de date. Modelul încearcă să ajusteze *anchor box*-urile pentru a se potrivi perfect cu aceste cutii *ground-truth*.

Concept Dificil: Offset Transformation (Regresia Dimensiunii)

- *Asta se întâmplă deoarece:* Deși cutiile ancoră se potrivesc aproximativ, modelul trebuie să învețe să le ajusteze. Dacă modelul ar prezice direct w și h (schimbarea lățimii și înălțimii), aceste valori ar fi foarte variabile. Prin utilizarea **logaritmului** transformăm raportul $\frac{w}{w_a}$ (raportul de mărime) într-o valoare care este aditivă și mai uniform distribuită, făcând sarcina de regresie (ajustarea dimensiunilor) mai stabilă și mai ușor de învățat.

Concept Dificil: Non-Maximum Suppression (NMS)

- **Analogie (Votul Multiplu):** Imaginați-vă că 100 de detectivi independenți propun cutii pentru locația unui suspect. Toate cutiile se suprapun și sunt foarte similare. NMS funcționează ca un moderator: el alege cutia detectivului cu cea mai mare încredere și le elimină pe toate celelalte care sunt prea apropiate de ea. Apoi, alege următoarea cea mai bună cutie (care nu a fost eliminată) și repetă, asigurându-se că fiecare obiect primește o singură reprezentare finală.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (IoU și Alocarea):**
  - **CAUZĂ:** Pentru a antrena modelul, trebuie să știm ce cutie ancoră se potrivește cel mai bine cu fiecare obiect real.
  - **EFECT:** IoU (măsura de similaritate) este folosită pentru a umple matricea $\mathbf{X}$ și a implementa **algoritmul de alocare** (assignment algorithm) care asociază cutiile *ground-truth* cu cutiile ancoră corespunzătoare, generând astfel etichetele de antrenament (clasa și offset-ul).
- **Implicații ale $wh(n+m-1)$:** Formula simplificată arată compromisul dintre complexitatea computațională și acoperirea spațiului. Deși $whnm$ acoperă toate combinațiile, $wh(n+m-1)$ este utilizat pentru a menține un număr mare de *anchor boxes* (necesar pentru acoperire), dar reducând numărul total de cutii propuse.

---

5. EXERCITII (Calcul și Analiză)

Exercițiul R.1: Calculul IoU (Intersection over Union)

Două cutii de delimitare sunt definite prin coordonatele colțurilor (Stânga Sus, Dreapta Jos):

- Cutia Ancoră A: $(x_{min} = 10, y_{min} = 10, x_{max} = 60, y_{max} = 60)$.
- Cutia Ground-Truth B: $(x_{min} = 30, y_{min} = 30, x_{max} = 70, y_{max} = 70)$.

Calculați valoarea IoU(A, B).

Rezolvare R.1:

1. **Calculul Ariilor Individiale:**
   - Aria A: $(60-10) \times (60-10) = 50 \times 50 = 2500$.
   - Aria B: $(70-30) \times (70-30) = 40 \times 40 = 1600$.
2. **Calculul Intersecției ($A \ \ B$):**
   - $x_{int_{min}} = \max(10, 30) = 30$.

- $y_{int_min} = \max(10, 30) = 30$.
- $x_{int_max} = \min(60, 70) = 60$.
- $y_{int_max} = \min(60, 70) = 60$.
- Lățime intersecție: $60 - 30 = 30$.
- Înălțime intersecție: $60 - 30 = 30$.
- Aria Intersecției: $30 \times 30 = \mathbf{900}$.

3. **Calculul Uniunii ($A \ B$):**

$$A \ B = A + B - A \ B$$

$$A \ B = 2500 + 1600 - 900 = 4100 - 900 = \mathbf{3200}$$

4. **Calculul IoU:**

$$o(A, B) = \frac{900}{3200} = \frac{9}{32} \approx \mathbf{0.28125}$$

Exercițiul R.2: Generarea Anchor Box (Formule de Dimensiune)

O imagine de intrare are dimensiunile $h = 600$ și $w = 800$. Se dorește generarea unei cutii ancore cu o scară $s = 0.5$ și un raport de aspect $r = 4$ (lățime/înălțime).

1. Calculați lățimea cutiei ancore (w_a).
2. Calculați înălțimea cutiei ancore (h_a).

Rezolvare R.2:

1. **Lățimea cutiei ancore (w_a):**

$$w_a = ws\sqrt{r} = 800 \times 0.5 \times \sqrt{4} = 400 \times 2 = \mathbf{800}$$

2. **Înălțimea cutiei ancore (h_a):**

$$h_a = hs/\sqrt{r} = 600 \times 0.5/\sqrt{4} = 300/2 = \mathbf{150}$$

Exercițiul R.3: Analiză Asupra Algoritmului de Alocare (Teoretic)

Descrieți de ce primul pas în algoritmul de alocare a cutiilor *ground-truth* (GT) la cutiile ancore implică găsirea elementului maxim $x_{i_1 j_1}$ în matricea IoU ($\mathbf{X}$) și eliminarea ulterioară a rândului și coloanei corespunzătoare.

Rezolvare R.3:

1. **Găsirea Maximului:** Găsirea celui mai mare element $x_{i_1 j_1}$ asigură că **cea mai bună potrivire** (IoU maxim) dintre *toate* cutiile ancore și *toate* cutiile *ground-truth* este selectată prima. Această alocare este intuitivă, deoarece A_{i_1} și B_{j_1} sunt cele mai apropiate perechi.

2. **Eliminarea:** Odată ce cutia *ground-truth* B_{j_1} a fost alocată cutiei ancore A_{i_1}, perechea este considerată rezolvată. Eliminarea rândului (i_1) și coloanei (j_1) din matricea $\mathbf{X}$ previne **realocarea** aceleiași cutii *ground-truth* sau a aceleiași cutii ancore în iterările ulterioare, asigurând o alocare unică și optimă inițială pentru fiecare cutie *ground-truth*.

6.5

Suntem la lecția **6.5 Multiscale object detection (Detecția Obiectelor pe Scări Multiple)**. Această lecție abordează provocarea de a detecta eficient obiecte de dimensiuni extrem de variate (mici vs. mari) fără a genera un număr prohibitiv de cutii ancoră. Soluția este utilizarea hărților de *feature*-uri (*feature maps*) la diferite scări.

Ghid de Studiu Complet: 6.5 Detecția Obiectelor pe Scări Multiple

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 6.5 introduce conceptul de detecție a obiectelor pe scări multiple ca răspuns la limitările detectării bazate pe cutiile ancoră generate pe fiecare pixel al imaginii de intrare. Deși în Secțiunea 6.4 s-a descris o metodă de generare a cutiilor ancoră centrate pe fiecare pixel, acest lucru poate duce la **prea multe cutii ancoră de calculat** (de exemplu, peste două milioane pentru o imagine de 561×728 cu cinci cutii pe pixel). Soluția adoptată este **eșantionarea uniformă a centrelor cutiilor ancoră** bazată pe forma hărților de *feature*-uri (*feature maps*) obținute din straturile convoluționale. Această abordare permite ajustarea numărului de cutii ancoră generate la diferite scări și exploatează capacitatea rețelelor neuronale profunde de a utiliza **reprezentările pe niveluri** pentru a detecta obiecte de diferite dimensiuni.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Problema:** Generarea cutiilor ancoră pe fiecare pixel al imaginii de intrare duce la un număr excesiv de mare de cutii de etichetat și prezis.
2. **Soluția de Eșantionare:** Centrele cutiilor ancoră sunt eșantionate uniform pe imaginea de intrare, determinând pozițiile lor pe baza formei (dimensiunii) unei **hărți de *feature*-uri** (*feature map*).
3. **Optimizarea Eșantionării:** Obiectele mai mici sunt mai susceptibile de a apărea în mai multe moduri decât obiectele mari; de aceea, se pot eșantiona mai multe regiuni pentru detectarea obiectelor mici și mai puține pentru cele mari.
4. **Mecanismul CNN:** Unitățile hărților de *feature*-uri la aceeași poziție spațială (dar cu canale diferite, c) au același câmp receptiv (*receptive field*), permițând modelului să utilizeze informația din acel câmp pentru a prezice clasa și *offset*-urile cutiilor ancoră.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.5 Detecția Obiectelor pe Scări Multiple

Eșantionarea și Reducerea Cutiilor Ancoră

- Metoda anterioară (Secțiunea 6.4) descria generarea cutiilor ancoră centrate pe fiecare pixel al unei imagini de intrare.
- Această metodă poate genera **prea multe cutii ancoră de calculat**.
- Exemplu numeric: O imagine de intrare de 561×728 cu **cinci** cutii ancoră pe pixel necesită etichetarea și predicția a **peste două milioane** de cutii ancoră ($561 \times 728 \times 5$).
- O metodă de reducere a numărului de cutii ancoră este eșantionarea uniformă a unei porțiuni mici de pixeli din imaginea de intrare pentru a genera cutiile ancoră centrate pe aceștia.
- La scări diferite, se pot genera **numere diferite de cutii ancoră de dimensiuni diferite**.
- Intuitiv, obiectele mai mici sunt mai susceptibile de a apărea pe o imagine decât obiectele mai mari.

Eșantionarea bazată pe Hărțile de Feature-uri

- O matrice bidimensională de ieșire a unui strat convoluțional se numește **hartă de *feature*-uri (*feature map*)**.
- Definind forma hărții de *feature*-uri, se pot determina centrele cutiilor ancoră eșantionate uniform pe imaginea de intrare.
- Cutiile ancoră sunt generate pe harta de *feature*-uri cu **fiecare unitate (pixel) ca centru al cutiei ancoră**.
- Dată fiind lățimea și înălțimea hărții de *feature*-uri, $fmap_w$ și $fmap_h$, se eșantionează uniform pixeli în $fmap_h$ rânduri și $fmap_w$ coloane pe imaginea de intrare.

Exemplu de Aplicare pe Scări Multiple (Vizualizări 4x4, 2x2, 1x1)

- **Detecția Obiectelor Mici (Scara 1):** Harta de *feature*-uri are dimensiunea 4×4. Centrele cutiilor ancoră sunt distribuite uniform în 4 rânduri și 4 coloane pe imagine. Scala cutiei ancoră este setată la **0.15**. Se observă că centrele cutiilor ancoră nu se suprapun.
- **Detecția Obiectelor Medii (Scara 2):** Înălțimea și lățimea hărții de *feature*-uri sunt **reduse la jumătate**. Se folosesc cutii ancoră mai mari cu scala setată la **0.4**. Unele cutii ancoră se suprapun.
- **Detecția Obiectelor Mari (Scara 3):** Înălțimea și lățimea hărții de *feature*-uri sunt **reduse din nou la jumătate**. Scala cutiei ancoră este mărită la **0.8**. Centrul cutiei ancoră devine **centrul imaginii** (când 1×1).

Mecanismul de Predicție

- La o anumită scară, dacă avem c hărți de *feature*-uri de forma $h \times w$, se generează hw seturi de cutii ancoră, fiecare set având același centru.
- Fiecare cutie ancoră este etichetată cu clasa și *offset*-ul bazat pe cutiile *ground-truth*.
- Modelul de detecție trebuie să prezică clasa și *offset*-ul celor hw seturi de cutii ancoră pe imaginea de intrare.
- Unitățile c ale hărților de *feature*-uri la aceeași poziție spațială au același **câmp receptiv** pe imaginea de intrare.
- Modelul transformă cele c unități ale hărților de *feature*-uri în clasele și *offset*-urile cutiilor ancoră generate folosind acea poziție spațială.
- Reprezentările pe niveluri ale imaginilor la multiple niveluri sunt utilizate pentru detecția pe scări multiple. Straturile de *feature maps* mai apropiate de stratul de ieșire au **câmpuri receptive mai largi** și pot detecta obiecte mai mari.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Câmp Receptiv (Receptive Field)

- **Analogie Simplificată (Fereastra de Vizualizare):** Câmpul receptiv al unui pixel pe o hartă de *feature*-uri este zona din imaginea de intrare pe care acel pixel o "vede" (a fost influențat de ea) în timpul procesării prin straturile convoluționale.
- *Asta se întâmplă deoarece:* Straturile superioare (mai aproape de ieșire) acumulează informații dintr-o zonă mai mare a imaginii de intrare, având un **câmp receptiv mai larg**. Acest lucru le face ideale pentru **detectarea obiectelor mari**. Straturile inferioare (hărți de *feature*-uri mai mari) au câmpuri receptive mai înguste, dar rezoluție spațială mai bună, fiind potrivite pentru **detectarea obiectelor mici**.

Concept Dificil: Eșantionarea Uniformă bazată pe Feature Maps

- **Explicație Simplificată:** În loc să punem cutii ancoră pe absolut fiecare pixel (ceea ce este ineficient), folosim rețeaua CNN pentru a ne spune unde să le punem. Dacă avem o hartă de *feature*-uri de 4×4, asta înseamnă că rețeaua a redus imaginea originală la 16 "puncte de interes" uniform distribuite. Noi generăm cutii ancoră doar centrate pe aceste 16 puncte. Dacă harta de *feature*-uri este 1×1, generăm cutii ancoră doar în centrul imaginii.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Scală și Eșantionare):**

- **CAUZĂ:** Obiectele mici sunt mai numeroase (apar în mai multe locuri, ex: 1×1 în 2×2 poate apărea de 4 ori).
- **EFECT:** Pentru detectarea obiectelor mici, folosim hărți de *feature*-uri cu rezoluție mare (ex: 4×4) pentru a eșantiona mai multe regiuni. Pentru obiectele mari, folosim hărți de *feature*-uri cu rezoluție mică (ex: 1×1) pentru a eșantiona mai puține regiuni (ex: doar centrul).

- **Conexiunea cu Capitolul 6.2 (Reprezentări pe Niveluri):** Metoda multiscale se bazează direct pe ideea introdusă la începutul capitolului: că reprezentările pe niveluri ale imaginii la multiple nivele pot fi valorificate de rețelele profunde pentru sarcini complexe.

---

5. EXERCITII (Calcul și Analiză)

Exercițiul M.1: Complexitatea Cutiilor Ancoră

O rețea de detecție folosește o imagine de intrare de 1024×1024 pixeli. Presupunând că se generează $k = 9$ cutii ancoră pe pixel (combinații de scări și rapoarte de aspect).

1. Calculați numărul total de cutii ancoră care trebuie etichetate și prezise dacă se folosește metoda descrisă în Secțiunea 6.4 (generare pe fiecare pixel).
2. Dacă se folosește o metodă pe scări multiple bazată pe o hartă de *feature*-uri de 8×8, câte seturi de cutii ancoră sunt generate, presupunând că $k = 9$ cutii ancoră sunt centrate pe fiecare unitate a *feature map*-ului?

Rezolvare M.1:

1. **Numărul Total de Anchor Boxes (Metoda 6.4):**

$$\text{Total oxes} = \text{ățime} \times \text{Înălțime} \times k$$

$$\text{Total oxes} = 1024 \times 1024 \times 9 = 1,048,576 \times 9 = \mathbf{9,437,184}$$

   (Aproximativ 9.4 milioane de cutii).
2. **Numărul Total de Anchor Boxes (Metoda Multiscale bazată pe 8×8 feature map):**

$$\text{Număr eturi Anchor oxes} = fmap_w \times fmap_h$$

$$\text{Număr eturi} = 8 \times 8 = 64 \text{ seturi}$$

$$\text{Total oxes} = 64 \times 9 = \mathbf{576}$$

   (Se observă o reducere drastică a numărului de cutii de analizat, de la milioane la doar 576, prin eșantionare bazată pe *feature maps*).

Exercițiul M.2: Relația Obiect-Mărime și Eșantionare (Conceptual)

Folosind exemplul dat în sursă despre apariția obiectelor (1×1 în 2×2 apare de 4 ori), explicați de ce se eșantionează mai multe regiuni (feature maps mai mari) pentru detectarea obiectelor mici.

Rezolvare M.2:

1. **Frecvența Obiectelor Mici:** Obiectele mici (1×1) au mai multe poziții posibile de apariție într-o regiune mai mare (2×2) (4 poziții posibile, conform sursei).
2. **Necesitatea Eșantionării Dense:** Pentru a captura toate aceste poziții posibile, este necesar să se eșantioneze mai multe regiuni, adică să se utilizeze o hartă de *feature*-uri cu o **rezoluție spațială mai mare** (ex: 4×4 sau 8×8).
3. **Contrastul cu Obiectele Mari:** Obiectele mari (2×2 în 2×2) au un singur mod de apariție, permițând utilizarea unei rezoluții spațiale mai scăzute (ex: 1×1) pentru a economisi resurse, deoarece ele pot fi detectate chiar și cu o eșantionare redusă la centru.

6.6

Lecția **6.6 Single shot multibox detection (SSD)** reprezintă un model complet și eficient de detecție a obiectelor. Bazându-se pe cunoștințele fundamentale despre cutiile de delimitare, cutiile ancoră și detecția pe scări multiple (Secțiunile 6.3–6.5), SSD integrează aceste principii într-o arhitectură unică, rapidă și utilizată pe scară largă.

Ghid de Studiu Complet: 6.6 Single shot multibox detection (SSD)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Modelul **Single Shot Multibox Detection (SSD)** este un model de detecție a obiectelor simplu, rapid și utilizat pe scară largă. Acesta reprezintă o implementare directă a principiilor de detecție pe scări multiple (Multiscale object detection). Arhitectura SSD constă dintr-o **rețea de bază (*base network*)** urmată de mai multe **blocuri de hărți de *feature*-uri pe scări multiple (*multiscale feature map blocks*)**. Rolul rețelei de bază este de a extrage *feature*-uri din imaginea de intrare, utilizând de obicei o rețea CNN profundă, cum ar fi VGG (trunchiată înainte de stratul de clasificare) sau ResNet. SSD generează un număr variabil de cutii ancoră de dimensiuni diferite și detectează obiecte de mărimi variabile prin prezicerea claselor și *offset*-urilor acestor cutii. Antrenarea modelului se realizează prin optimizarea unei **funcții de pierdere compuse** (suma pierderii de clasificare și a pierderii de regresie).

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectura:** SSD utilizează o rețea de bază (ex. VGG, ResNet) urmată de blocuri de

feature map pe scări multiple.

2. **Mecanismul de Predicție:** Clasa și predicția cutiei de delimitare (*offset*) sunt realizate folosind **canalele straturilor convoluționale** pentru a reduce complexitatea modelului, în loc de straturi complet conectate.
3. **Pierderea (Loss Function):** Funcția de pierdere este o sumă a **pierderii de clasificare** (entropie încrucișată) și a **pierderii de regresie a *offset*-urilor** (norma ℓ_1 - eroarea absolută).
4. **Detectarea pe Scări:** Hărțile de *feature*-uri mai mici (mai aproape de vârful arhitecturii) au **câmpuri receptive mai mari** și sunt folosite pentru detectarea obiectelor **mai mari**.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.6 Single shot multibox detection (SSD) – Arhitectura și Designul

- **Fundament:** SSD este un model de detecție a obiectelor pe scări multiple.
- **Componente Principale (Figura 4):** Un model SSD constă dintr-o rețea de bază, urmată de mai multe blocuri de hărți de *feature*-uri pe scări multiple.
- **Rețeaua de Bază (*Base Network*):** Extrage *feature*-uri din imaginea de intrare. Poate fi o rețea CNN profundă.
  - Exemple: Rețeaua VGG trunchiată înainte de stratul de clasificare, sau ResNet.
  - Prin proiectare, rețeaua de bază poate scoate hărți de *feature*-uri mai mari pentru a genera mai multe cutii ancoră pentru detectarea obiectelor mai mici.
- **Blocuri Multiscale Feature Map:** Aceste blocuri reduc succesiv (ex., la jumătate) înălțimea și lățimea hărților de *feature*-uri din blocul anterior, mărind câmpul receptiv al fiecărei unități pe imaginea de intrare.
- **Set de Cinci Blocuri:** Modelul SSD complet constă în cinci blocuri:
  - Primul bloc: Rețeaua de bază.
  - Al doilea până la al patrulea bloc: Blocuri de *downsampling* (reduc dimensiunea la jumătate).
  - Al cincilea bloc: Folosește *global maximum pooling* pentru a reduce înălțimea și lățimea la **1**.
- **Utilizarea Blocului:** Hărțile de *feature*-uri produse de fiecare bloc sunt utilizate pentru (i) generarea cutiilor ancoră și (ii) prezicerea claselor și *offset*-urilor acestor cutii.
- **Corelație cu Scara Obiectelor:** Hărțile de *feature*-uri mai mici (mai aproape de vârful Figurii 4) au câmpuri receptive mai mari, fiind potrivite pentru detectarea obiectelor **mai mari**.

Implementarea Predicției (Clase și Bounding Boxes)

- **Clasificarea:** Dacă q este numărul de clase de obiecte, cutiile ancoră trebuie să fie clasificate în $q+1$ **clase** (clasa 0 fiind *background*).
- **Reducerea Complexității:** SSD utilizează **canalele straturilor convoluționale** pentru a prezice clasele, deoarece utilizarea straturilor complet conectate ar face costul de parametrizare prohibitiv.
- **Stratul de Predicție a Clasei:** Folosește un strat convoluțional de 3×3 cu *padding* de 1, care nu modifică lățimea sau înălțimea hărților de *feature*-uri.
- **Numărul de Canale de Ieșire (Clase):** Pentru a produce predicții valide, numărul de canale de ieșire trebuie să fie $a(q+1)$, unde a este numărul de cutii ancoră generate pe poziție spațială.
  - Un canal de ieșire cu indexul $i(q+1)+j$ reprezintă predicția clasei j pentru cutia ancoră i.
- **Stratul de Predicție a Bounding Box:** Are un design similar cu cel al stratului de predicție a clasei, dar numărul de ieșiri este diferit: trebuie să prezică **patru *offset*-uri** (nu $q+1$ clase).

Designul Blocului de Downsampling și Scalele

- **Blocul de Downsampling:** Reduce la jumătate înălțimea și lățimea hărților de *feature*-uri.
  - Design: Două straturi convoluționale de 3×3 cu *padding* de 1, urmate de un strat de *maximum pooling* de 2×2 cu *stride* 2.
  - Efect: Blocurile de *downsampling* măresc câmpul receptiv al fiecărei unități în harta de *feature*-uri de ieșire.
- **Exemplu Numeric de Bază:** O imagine de 256×256 intră în rețeaua de bază (formată din trei blocuri de *downsampling*), rezultând o hartă de *feature*-uri de 32×32 ($256/2^3 = 32$).
- **Scalele Cutiilor Ancoră:** Intervalul de la 0.2 la 1.05 este împărțit în cinci secțiuni pentru a determina valorile scărilor mici la cele cinci blocuri: **0.2, 0.37, 0.54, 0.71, și 0.88**. Valorile scărilor mari sunt date de $\sqrt{s_{min} \cdot s_{max}}$.

Antrenarea și Evaluarea Modelului

- **Funcția de Pierdere (Loss):** Detecția obiectelor are **două tipuri de pierderi**:
  1. **Pierderea de Clasificare:** Utilizează funcția de pierdere a entropiei încrucișate (*cross-entropy loss*).
  2. **Pierderea de Regresie a Offset-urilor:** Pentru cutiile ancoră pozitive (non-background), este folosită **norma ℓ_1 loss** (eroarea absolută), nu pierderea pătratică (*squared loss*).
     - Pierderea totală este suma pierderii de clasificare a cutiilor ancoră și a pierderii de *offset* a cutiilor ancoră.
     - Cutiile ancoră negative (*background*) și cele ilegale (*padded*) sunt filtrate din calculul pierderii.

- **Evaluarea:**
  - Rezultatele clasificării sunt evaluate folosind **acuratețea (*accuracy*)**.
  - Rezultatele predicției *bounding box*-urilor sunt evaluate folosind **eroarea medie absolută (*mean absolute error*)**, datorită utilizării normei ℓ_1 loss.

Predicția Finală

- Cutia de delimitare prezisă este obținută din cutiile ancoră și *offset*-urile prezise.
- Se aplică **Suprimarea Non-Maximă (NMS)** pentru a elimina cutiile de delimitare prezise similare și suprapuse.
- Se afișează cutiile prezise care au un scor de încredere (*confidence*) de **0.9 sau mai mult**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Single Shot (SSD)

- **Explicație Simplificată:** "Single Shot" înseamnă că modelul face clasificarea și localizarea (regresia cutiei de delimitare) **într-o singură trecere** a rețelei (spre deosebire de modelele *two-stage* precum R-CNN, care necesită mai multe etape). Acest lucru face modelul rapid.

Concept Dificil: Rolul Pierderii ℓ_1 (Norma Absolută)

- *Asta se întâmplă deoarece:* În regresia *offset*-urilor, pot exista erori mari (outlieri) în predicția cutiilor. Pierderea pătratică (squared loss) ar amplifica aceste erori mari (le-ar ridica la pătrat), făcând antrenamentul instabil. **Norma** ℓ_1 (eroarea absolută) tratează erorile mari liniar, făcând modelul mai robust la *outlier*-i și, prin urmare, mai potrivit pentru sarcina de regresie a cutiei de delimitare.

Concept Dificil: Predicția prin Canalele Convoluționale

- **Analogie:** Imaginați-vă că harta de *feature*-uri este o tablă de șah 8×8. Fiecare pătrat este un set de cutii ancoră. În loc să luăm datele din fiecare pătrat și să le dăm unui strat complet conectat (MLP) care să prezică clasele (costisitor), noi aplicăm un mic filtru convoluțional (3×3). Ieșirea acestui filtru, pentru fiecare canal, este direct predicția pentru o anumită cutie ancoră/clasă. Acest lucru reduce dramatic costul de parametrizare prin **partajarea ponderilor** (*weight sharing*) peste toate pozițiile spațiale.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Secțiunea 6.5:** SSD este un exemplu practic de implementare a **detecției pe scări multiple** prin utilizarea hărților de *feature*-uri de diferite dimensiuni (de la 32×32 la 1×1).
- **Conexiunea cu Secțiunea 6.4:** SSD se bazează pe principiul că hărțile de *feature*-uri la o anumită poziție ($h \times w$) sunt folosite pentru a genera cutii ancoră și pentru a prezice clasa și *offset*-ul pentru acele cutii. De asemenea, utilizează **NMS** la final pentru a curăța predicțiile.
- **Relația Cauză-Efect (Adâncimea Rețelei):** Blocurile de *downsampling* (cauză) măresc câmpul receptiv (efect). Hărțile de *feature*-uri mai mici rezultate (cauză) sunt mai potrivite pentru detectarea obiectelor mari (efect).

---

5. EXERCITII (Calcul și Analiză Dimensională)

Exercițiul S.1: Calculul Total al Cutiilor Ancoră (SSD)

Se folosește modelul SSD complet, care constă în cinci blocuri (cu *feature maps* de $32^2, 16^2, 8^2, 4^2, 1^2$). Se asumă că la fiecare scară se generează $a = 4$ cutii ancoră pe unitatea spațială.

Calculați numărul total de cutii ancoră generate de modelul SSD pentru o singură imagine, utilizând suma dimensiunilor spațiale ale hărților de *feature*-uri.

Rezolvare S.1:

Formula este $\text{Total oxes} = \quad {}_{i=1}^{5}(h_i \cdot w_i) \cdot a.$

$$\text{Total oxes} = (32^2 + 16^2 + 8^2 + 4^2 + 1^2) \times 4$$

$$\text{Total oxes} = (1024 + 256 + 64 + 16 + 1) \times 4$$

$$\text{Total oxes} = 1361 \times 4 = \mathbf{5444}$$

(Acest rezultat este confirmat de sursă).

Exercițiul S.2: Calculul Canalelor de Ieșire (Predicția Clasei)

Un bloc SSD utilizează o hartă de *feature*-uri de 8×8. Se dorește detectarea a $q = 5$ clase de obiecte, plus clasa *background*. Modelul generează $a = 6$ cutii ancoră pe poziție spațială.

1. Câte clase totale ($q + 1$) trebuie prezise?
2. Calculați numărul total de canale de ieșire (output channels) pe care trebuie să le aibă stratul de predicție a clasei pentru acea hartă de *feature*-uri.

Rezolvare S.2:

1. **Numărul Total de Clase:**

$$q + 1 = 5 + 1 = \mathbf{6} \text{ clase}$$

2. **Numărul Total de Canale de Ieșire:** Numărul de canale de ieșire este $a(q+1)$.

$$\text{Output hannels} = 6 \times (5+1) = 6 \times 6 = \mathbf{36}$$

(Așadar, harta de *feature*-uri de ieșire va avea dimensiunea $8 \times 8 \times 36$).

Exercițiul S.3: Analiză Asupra Pierderii (Teoretic)

De ce funcția de pierdere totală în SSD este compusă din două componente separate (una pentru clasificare și una pentru regresie), și care este cea mai importantă diferență în calculul pierderii de regresie față de metodele standard (ex: eroarea pătratică)?

Rezolvare S.3:

1. **Necesitatea Pierderii Compuse:** Sarcina de detecție a obiectelor este o problemă hibridă: necesită **Clasificare** (Ce este obiectul? Clasa) și **Regresie** (Unde este obiectul? Offset-ul). Prin urmare, pierderea trebuie să combine entropia încrucișată (pentru clasificare) și o pierdere de regresie (pentru *offset*-uri).
2. **Diferența în Regresie:** SSD folosește **norma ℓ_1 loss** (eroarea medie absolută) în locul erorii pătratice. Norma ℓ_1 este preferată deoarece este **mai robustă la *outlier*-i** (erori mari în predicția cutiilor) și menține antrenamentul mai stabil, tratând diferențele mari liniar, nu pătratic.

6.7

Lecția **6.7 Region-based CNNs (R-CNNs)** detaliază o serie de abordări inovatoare, bazate pe rețele convoluționale (CNN), care au deschis drumul spre detecția obiectelor de înaltă precizie. Aceste modele, denumite R-CNN (Region-based CNNs), reprezintă o clasă de metode pioniere, diferite de abordările *single shot* (precum SSD), care au evoluat de la modelul inițial R-CNN la variante mult mai rapide și mai avansate, culminând cu Mask R-CNN.

Ghid de Studiu Complet: 6.7 Region-based CNNs (R-CNNs)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Modelele **R-CNN** sunt abordări pioniere în detecția obiectelor care operează într-o manieră secvențială, de tip *two-stage* (două etape), spre deosebire de metodele *single shot*. Ideea centrală este generarea de **propuneri de regiuni (*region proposals*)** (similare cutiilor ancoră). Modelul inițial R-CNN este lent din cauza costului masiv de calcul: extrage *feature*-uri printr-o propagare forward CNN **independentă** pentru fiecare propunere de regiune. Îmbunătățirile ulterioare, **Fast R-CNN** și **Faster R-CNN**, au rezolvat această problemă prin **partajarea calculului CNN** pe întreaga imagine de intrare. **Faster R-CNN** a îmbunătățit

viteza și mai mult prin înlocuirea metodei lente *selective search* cu o **Rețea de Propuneri de Regiuni (Region Proposal Network - RPN)**. În cele din urmă, **Mask R-CNN** a extins Faster R-CNN pentru a efectua **Segmentare Semantică** la nivel de pixel.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Principiul R-CNN (Two-Stage):** Extragerea propunerilor de regiuni (ex. 2000).
2. **Problema R-CNN:** Viteză redusă din cauza propagării CNN forward **repetate** și independente pentru fiecare propunere de regiune.
3. **Soluția Fast R-CNN:** Partajarea calculului CNN: propagarea CNN forward este efectuată **doar pe întreaga imagine**. Utilizează stratul **Region of Interest (RoI) pooling**.
4. **Soluția Faster R-CNN:** Înlocuiește *selective search* cu o **Region Proposal Network (RPN)** care folosește cutii ancoră pentru a genera propuneri de regiuni de înaltă calitate.
5. **Mask R-CNN:** Extinde Faster R-CNN prin înlocuirea RoI pooling cu **RoI alignment** și adăugarea unei rețele convoluționale suplimentare pentru a prezice poziția obiectului la nivel de **pixel** (Segmentare).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

6.7.1 R-CNN (Original)

- R-CNN este o abordare pionieră de aplicare a *deep learning* în detecția obiectelor.
- **Etapa 1: Extragerea Propunerilor de Regiuni:** R-CNN extrage mai întâi multe (de exemplu, 2000) propuneri de regiuni (*region proposals*) din imaginea de intrare, adesea utilizând algoritmul **selective search**. Aceste regiuni sunt selectate la multiple scări, cu forme și dimensiuni diferite.
- **Etichetare:** Fiecare propunere de regiune este etichetată cu o clasă și o cutie de delimitare *ground-truth*.
- **Etapa 2: Extragerea Feature-urilor:** Un CNN pre-antrenat este ales (și trunchiat înainte de stratul de ieșire).
  - Fiecare propunere de regiune este **redimensionată** la dimensiunea de intrare a CNN-ului.
  - CNN-ul efectuează **propagare forward pe fiecare propunere de regiune** pentru a extrage *feature*-uri.
- **Etapa 3: Clasificare:** *Feature*-urile extrase sunt folosite pentru a antrena **multiple mașini cu vector suport (SVM)** pentru a clasifica obiectele (un SVM determină dacă o anumită clasă este prezentă).
- **Etapa 4: Regresia Cutiei de Delimitare:** *Feature*-urile sunt folosite și pentru a antrena un **model de regresie liniară** pentru a prezice cutia de delimitare *ground-truth*.

6.7.2 Dezavantajele R-CNN (Viteza)

- Modelul R-CNN este **lent**.
- **Cauza lentorii:** Selectarea a mii de propuneri de regiuni necesită **mii de propagări forward CNN** pe o singură imagine de intrare.
- **Blocaj de performanță:** Sarcina de calcul masivă este cauzată de propagarea CNN forward **independentă** pentru fiecare propunere de regiune, fără a partaja calculul.
- Suprapunerile dintre regiuni duc la **calcul repetat** al *feature*-urilor.

6.7.3 Fast R-CNN (Îmbunătățirea Vitezei)

- **Îmbunătățire Majoră:** CNN-ul efectuează propagarea forward **doar pe întreaga imagine**, nu pe propuneri individuale.
- **Arhitectura (Figura 6):**
  1. *Selective search* generează n propuneri de regiuni (ROI).
  2. CNN-ul procesează **întreaga imagine** pentru a extrage o hartă de *feature*-uri.
  3. Stratul **Region of Interest (RoI) pooling** ia ca intrare harta de *feature*-uri CNN și propunerile de regiuni.
  4. RoI pooling extrage *feature*-uri de **aceeași formă** ($h_2 \times w_2$) pentru toate propunerile de regiuni, chiar dacă regiunile de interes au forme diferite.
  5. *Feature*-urile concatenate sunt transformate printr-un strat complet conectat (FC).
  6. Clasificarea (folosind **softmax regression**) și predicția cutiei de delimitare ($n \times 4$) se fac din ieșirea stratului FC.
- **Diferența RoI Pooling vs. Pooling Standard:** Stratul RoI pooling permite **specificarea directă a formei de ieșire** ($h_2 \times w_2$), spre deosebire de stratul de *pooling* standard, unde forma de ieșire este controlată indirect prin *pooling window*, *padding* și *stride*.
- **Mecanismul RoI Pooling:** Regiunea de interes de forma $h \times w$ este împărțită într-o grilă de $h_2 \times w_2$ sub-ferestre. Cel mai mare element (*maximum*) este folosit ca ieșire a sub-ferestrei. (Exemplu: 3×3 transformat în 2×2 ieșire).

6.7.4 Faster R-CNN (Îmbunătățirea Propunerii de Regiuni)

- **Problema Fast R-CNN:** Deși mai rapid, încă se baza pe *selective search* pentru a genera propuneri de regiuni, care este lent și inexact.
- **Îmbunătățire:** Faster R-CNN înlocuiește *selective search* cu o **Region Proposal Network (RPN)** (Figura 8).
- **RPN Funcționare (Pași):**
  1. Se folosește un strat convoluțional de 3×3 cu *padding* de 1 pentru a transforma ieșirea CNN într-o nouă hartă de *feature*-uri cu c canale. Fiecare unitate spațială primește un nou vector de *feature*-uri de lungime c.
  2. Se generează multiple **cutii ancoră** (de scări și rapoarte de aspect diferite) centrate pe fiecare pixel al hărților de *feature*-uri.
  3. Folosind vectorul de *feature*-uri de lungime c la centrul fiecărei cutii ancoră, se prezice **clasa binară** (obiect sau *background*) și **bounding box** pentru acea cutie.

4. Se aplică **Non-Maximum Suppression (NMS)** pentru a elimina rezultatele suprapuse. Cutiile prezise rămase sunt **propunerile de regiuni** necesare stratului RoI pooling.

- **Antrenare Îmbinată:** RPN este antrenată **împreună (*jointly trained*)** cu restul modelului. Funcția obiectiv include nu doar predicția clasei și *bounding box*-ului pentru detecția finală, ci și predicția clasei binare și a *bounding box*-ului pentru cutiile ancoră din RPN.
- **Rezultat:** RPN învață să genereze propuneri de regiuni de înaltă calitate cu un **număr redus** de propuneri învățate din date.

6.7.5 Mask R-CNN (Segmentare la Nivel de Pixel)

- **Scop:** Îmbunătățirea preciziei detecției obiectelor prin exploatarea etichetelor detaliate la nivel de pixel (dacă sunt disponibile în setul de antrenament).
- **Modificare Arhitecturală:** Mask R-CNN modifică Faster R-CNN prin înlocuirea stratului RoI pooling cu stratul **Region of Interest (RoI) alignment** (Figura 9).
- **RoI Alignment:** Acest strat utilizează **interpolarea biliniară** pentru a păstra informațiile spațiale pe hărțile de *feature*-uri. Este mai potrivit pentru predicția la nivel de pixel (*pixel-level prediction*).
- **Ieșirea:** RoI alignment produce *feature maps* de aceeași formă pentru toate regiunile de interes.
- **Mecanism de Segmentare:** Pe lângă predicția clasei și a *bounding box*-ului, un **strat convoluțional complet conectat (*fully convolutional network*) suplimentar** este adăugat pentru a prezice **poziția obiectului la nivel de pixel (masca)**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: RoI Pooling (Region of Interest Pooling)

- **Analogie Simplificată (Ajustarea Dimensiunii):** Imaginați-vă că extragem din harta de *feature*-uri 100 de regiuni propuse, unele de 13×17 pixeli, altele de 20×20. RoI pooling ia aceste regiuni de dimensiuni diferite și le **forțează** să devină toate de aceeași dimensiune fixă (ex: 7×7), folosind *max pooling* în sub-ferestre. Această uniformizare este necesară pentru a alimenta straturile complet conectate ulterioare (FC).

Concept Dificil: Blocajul de Performanță al R-CNN (Calcul Repetat)

- *Asta se întâmplă deoarece:* Deoarece R-CNN extrage mii de propuneri de regiuni, și multe dintre ele se **suprapun** (ex: o propunere conține o parte din obiect, iar o altă propunere conține aceeași parte plus puțin fundal), rețeaua CNN recalculează aceleași

feature-uri de bază de mii de ori. **Fast R-CNN** rezolvă acest lucru rulând CNN-ul o singură dată pe întreaga imagine și refolosind *feature*-urile extrase.

Concept Dificil: RoI Alignment vs. RoI Pooling (Mask R-CNN)

- **Explicație Simplificată:** RoI pooling, prin utilizarea operației *max pooling*, poate introduce **erori de rotunjire (quantization error)** atunci când regiunile sunt de dimensiuni non-standard. De exemplu, împărțirea unei regiuni de 13×13 la o grilă de 7×7 implică rotunjirea dimensiunilor sub-ferestrelor. Pentru Segmentarea Semantică (Mask R-CNN), unde este necesară precizia la nivel de pixel, aceste erori de rotunjire sunt inacceptabile. **RoI Alignment** (utilizând interpolarea biliniară) menține precizia sub-pixelică, fiind mult mai potrivit pentru prezicerea măștilor detaliate.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Evoluția Vitezei):**
  - **R-CNN (CAUZĂ):** Propagare CNN repetată de mii de ori. **(EFECT):** Model lent, nefezabil în aplicațiile în timp real.
  - **Fast R-CNN (CAUZĂ):** Partajarea calculului CNN pe întreaga imagine. **(EFECT):** Viteză semnificativ îmbunătățită.
  - **Faster R-CNN (CAUZĂ):** Înlocuirea *selective search* cu RPN (o rețea neuronală rapidă). **(EFECT):** Generarea propunerilor de regiuni devine mult mai rapidă, crescând viteza generală a modelului.
- **Conexiunea cu Secțiunea 6.4 (Anchor Boxes):** RPN-ul introdus în Faster R-CNN se bazează pe conceptul de **cutii ancoră** pentru a prezice clasele binare și *bounding box*-urile.

---

5. EXERCITII (Calcul și Analiză Dimensională)

Exercițiul R.4: Calculul Frecvenței de Propagare (R-CNN vs. Fast R-CNN)

Presupuneți o sarcină de detecție a obiectelor pe o secvență de 10 imagini. Modelul *selective search* generează $_{prop} = 2000$ propuneri de regiuni per imagine.

1. Calculați de câte ori trebuie rulată propagarea CNN forward (extragerea de *feature*-uri) în total pe secvența de 10 imagini, dacă se utilizează modelul R-CNN inițial.
2. Calculați de câte ori trebuie rulată propagarea CNN forward în total pe secvența de 10 imagini, dacă se utilizează modelul Fast R-CNN.

Rezolvare R.4:

1. **R-CNN inițial:** CNN rulează pentru *fiecare* propunere de regiune pe *fiecare* imagine.

$$\text{Total Propaări} = \text{Număr Imaini} \times \text{Număr Propuneri per Imaine}$$

$$\text{Total Propaări} = 10 \times 2000 = \mathbf{20,000}$$

2. **Fast R-CNN:** CNN rulează **o singură dată** pe întreaga imagine, iar *feature*-urile sunt apoi refolosite prin RoI pooling.

$$\text{Total Propaări} = \text{Număr Imaini} \times 1 = 10 \times 1 = \mathbf{10}$$

(Diferența majoră (20.000 vs. 10) ilustrează de ce R-CNN este lent și Fast R-CNN este mult mai rapid).

Exercițiul R.5: Analiza Dimensională a RoI Pooling (Fast R-CNN)

Un strat convoluțional al unui Fast R-CNN produce o hartă de *feature*-uri de $256 \times c \times 32 \times 32$ (unde 256 este *batch size*, c este numărul de canale). Stratul RoI pooling este configurat pentru a extrage *feature*-uri de formă $h_2 = 7$ și $w_2 = 7$.

1. Dacă *selective search* a generat $n = 1000$ propuneri de regiuni pentru imaginea curentă, care va fi forma *feature*-urilor concatenate de ieșire din stratul RoI pooling?
2. Câte elemente conține fiecare sub-fereastră (subwindow) în exemplul ilustrativ al RoI pooling din sursă (pentru un input de 3×3 transformat în 2×2 output)?

Rezolvare R.5:

1. **Forma ieșirii RoI Pooling:** Ieșirea RoI pooling are forma $n \times c \times h_2 \times w_2$.

$$\text{Forma Ieșirii} = \mathbf{1000} \times \mathbf{c} \times \mathbf{7} \times \mathbf{7}$$

2. **Elementele Sub-ferestrei (Exemplul 3×3 la 2×2):** Sursa indică (pentru o regiune 3×3 transformată în 2×2):
   - Sub-fereastra Stânga Sus conține elementele **0, 1, 4, și 5** (maximul este 5).
   - Sub-fereastra Dreapta Sus conține elementele **2 și 6** (maximul este 6).
   - Sub-fereastra Stânga Jos conține elementele **8 și 9** (maximul este 9).
   - Sub-fereastra Dreapta Jos conține elementele **10**. *(Notă: Sursa menționează doar 10, dar în figura 7 ar trebui să includă și 11, deși 10 este valoarea maximă în acea zonă de 3×3)*.

6.8

Înțeleg perfect cerința. Având în vedere că secțiunea de viziune computerizată (Capitolele 6.1–6.7) necesită o înțelegere solidă a terminologiei, procedurilor și a evoluției modelelor, voi genera un set complex de exerciții sub formă de întrebări și răspunsuri, acoperind toate subiectele menționate (Image Augmentation, Fine-Tuning, Bounding Boxes, Anchor Boxes, Multiscale Detection, și R-CNNs).

Vom structura exercițiile în trei părți: **I. Sinteză și Diferențe (Teorie de Bază)**, **II. Analiză Procedurală și Evoluție (R-CNNs)** și **III. Calcul Aplicat (Formule și Dimensiuni)**.

Exerciții Tip Examen (Bac la Română - Viziune Computerizată)

I. Sinteză și Diferențe (Teorie de Bază)

Întrebarea 1 (Object Detection & Bounding Boxes):

Explicați diferența fundamentală dintre sarcina de **Clasificare a Imaginilor** (Capitolul 3) și sarcina de **Detecție a Obiectelor** (Capitolul 6.3). Cum este descrisă localizarea spațială a obiectului în detecție și care este sistemul de coordonate utilizat?

Răspuns 1:

Sarcina de **Clasificare a Imaginilor** presupune de obicei prezența unui singur obiect major în imagine și are ca scop identificarea categoriei acelui obiect [conversation history, Secțiunea 6.3]. Sarcina de **Detecție a Obiectelor** este mai complexă, deoarece necesită nu doar identificarea categoriei, ci și determinarea **pozițiilor specifice** ale **multiplelor obiecte** de interes dintr-o singură imagine [conversation history, Secțiunea 6.3].

Localizarea spațială a obiectului este descrisă prin **Cutia de Delimitare (*Bounding Box*)**, care este dreptunghiulară [conversation history, Secțiunea 6.3]. Sistemul de coordonate utilizat în imagine are originea $(0, 0)$ în **colțul din stânga sus**, cu axa x pozitivă spre dreapta și axa y pozitivă **în jos** [conversation history, Secțiunea 6.3].

Întrebarea 2 (Image Augmentation & Fine-Tuning):

Numiți și explicați pe scurt cele două tehnici introduse la începutul Capitolului 6 pentru îmbunătățirea generalizării modelului. Care este principalul beneficiu al Fine-Tuning-ului?

Răspuns 2:

Cele două tehnici sunt:

1. **Augmentarea Imaginilor (*Image augmentation*):** Generarea de exemple de antrenament similare, dar distincte, prin aplicarea de schimbări aleatorii (ex. răsturnare pe orizontală, ajustarea luminozității, decupare aleatorie) [conversation history, Secțiunea 6.1]. Aceasta extinde setul de date și reduce dependența modelului de atribute specifice [conversation history, Secțiunea 6.1].
2. **Fine-Tuning-ul:** O tehnică de *transfer learning* care folosește un model pre-antrenat pe un set de date mare (ex. ImageNet) și transferă cunoștințele (feature-uri generale) către o sarcină țintă mai mică [conversation history, Secțiunea 6.2].

Principalul beneficiu al Fine-Tuning-ului este că permite obținerea de performanțe ridicate chiar și pe seturi de date țintă mult mai mici, **evitând costul și timpul necesar** pentru colectarea și etichetarea unui set de date masiv de la zero [conversation history, Secțiunea 6.2].

Întrebarea 3 (Anchor Boxes & Multiscale Detection):

Definiți rolul **Cutiilor Ancoră (*Anchor boxes*)** și explicați de ce metoda **Detecției pe Scări Multiple (*Multiscale object detection*)** este necesară în practică.

Răspuns 3:

Cutiile Ancoră sunt cutii de delimitare generate cu scări și rapoarte de aspect variabile, centrate pe fiecare poziție spațială eșantionată [conversation history, Secțiunea 6.4]. Ele oferă modelului un set de propuneri inițiale pe care acesta le va ajusta (prin *offset*) și clasifica.

Detecția pe Scări Multiple este necesară deoarece generarea cutiilor ancoră pe fiecare pixel al imaginii de intrare (metoda simplă) duce la un număr **prohibit de mare** de cutii ancoră de procesat (ex. peste două milioane pentru o imagine 561×728) [conversation history, Secțiunea 6.5]. Soluția este **eșantionarea uniformă a centrelor cutiilor ancoră** bazată pe forma hărților de *feature*-uri (*feature maps*), permițând eșantionarea diferită pentru detectarea obiectelor mici (rezoluție înaltă) și mari (rezoluție scăzută/câmp receptiv larg) [conversation history, Secțiunea 6.5].

II. Analiză Procedurală și Evoluție (R-CNNs)

Întrebarea 4 (Arhitectura R-CNN):

Descrieți cei patru pași principali ai modelului R-CNN inițial (Figura 5) și identificați **principalul blocaj de performanță** al acestei arhitecturi.

Răspuns 4:

Modelul R-CNN inițial (Region-based CNN) implică patru pași principali:

1. Se extrag multe (ex. 2000) **propuneri de regiuni** (*region proposals*) din imaginea de intrare, folosind adesea algoritmul **selective search**. Aceste propuneri sunt etichetate cu o clasă și o cutie de delimitare *ground-truth*.
2. Se alege un CNN pre-antrenat, trunchiat înainte de stratul de ieșire. Fiecare propunere de regiune este redimensionată la dimensiunea de intrare a CNN-ului, iar rețeaua execută **propagare forward** pentru a extrage *feature*-uri.
3. *Feature*-urile extrase sunt folosite pentru a antrena **multiple mașini cu vector suport (SVM)** pentru a clasifica obiectele (o SVM pentru fiecare clasă specifică).

4. *Feature*-urile sunt folosite și pentru a antrena un **model de regresie liniară** pentru a prezice cutia de delimitare *ground-truth*.

Principalul blocaj de performanță al R-CNN este **propagarea forward CNN independentă** pentru fiecare propunere de regiune, fără partajarea calculului. Acest lucru face ca modelul să fie lent, deoarece generează **mii de propagări CNN** pe o singură imagine, rezultând mult calcul repetat din cauza suprapunerii regiunilor.

Întrebarea 5 (Evoluția R-CNN):

Explicați cum a rezolvat **Fast R-CNN** problema de performanță a R-CNN și ce strat nou a introdus (Figura 6). Apoi, explicați cum a îmbunătățit **Faster R-CNN** această arhitectură (Figura 8).

Răspuns 5:

Fast R-CNN a rezolvat problema de performanță prin efectuarea propagării CNN forward **o singură dată pe întreaga imagine**, spre deosebire de R-CNN care rula CNN-ul pe fiecare propunere individuală.

Stratul nou introdus este **Region of Interest (RoI) pooling layer**. RoI pooling ia harta de *feature*-uri obținută din CNN și propunerile de regiuni și extrage *feature*-uri de **aceeași formă** (specificată, ex. $h_2 \times w_2$) pentru toate propunerile de regiuni, permițând concatenarea lor și alimentarea straturilor complet conectate (FC).

Faster R-CNN a îmbunătățit Fast R-CNN prin înlocuirea metodei lente *selective search* cu o **Region Proposal Network (RPN)**. RPN utilizează un strat convoluțional, generează **cutii ancoră** de diferite scări și rapoarte de aspect, și prezice **clasa binară** (obiect sau *background*) și *bounding box* pentru aceste cutii ancoră. Predicțiile RPN, după aplicarea NMS (Non-Maximum Suppression), devin propunerile de regiuni de înaltă calitate necesare stratului RoI pooling.

Întrebarea 6 (Mask R-CNN):

Ce tip de sarcină extinsă abordează **Mask R-CNN** și prin ce modificare arhitecturală esențială se diferențiază de Faster R-CNN (Figura 9)?

Răspuns 6:

Mask R-CNN abordează sarcina de **Segmentare la Nivel de Pixel** (sau Segmentare Semantică, atunci când etichetele la nivel de pixel sunt disponibile).

Mask R-CNN se modifică față de Faster R-CNN înlocuind stratul **Region of Interest (RoI) pooling** cu stratul **Region of Interest (RoI) alignment**. RoI alignment folosește **interpolarea biliniară** pentru a păstra informațiile spațiale pe hărțile de *feature*-uri, ceea ce este mai potrivit pentru predicția la nivel de pixel. De asemenea, Mask R-CNN adaugă o

rețea convoluțională suplimentară (*fully convolutional network*) pentru a prezice poziția obiectului la nivel de pixel (masca), pe lângă predicția clasei și a *bounding box*-ului.

III. Calcul Aplicat (Formule și Dimensiuni)

Exercițiul A.1 (RoI Pooling):

Se consideră un strat RoI pooling (Fast R-CNN) care primește o regiune de interes de formă $h = 10$ și $w = 10$ de pe harta de *feature*-uri a CNN-ului. Se specifică faptul că ieșirea dorită este de formă $h_2 = 5$ și $w_2 = 5$.

1. Calculați dimensiunile aproximative ale fiecărei sub-ferestre (*subwindow*) în grila $h_2 \times w_2$.
2. Dacă ieșirea RoI pooling este 5×5 și numărul de canale c este 256, care va fi numărul total de *feature*-uri extras pentru o singură propunere de regiune?

Rezolvare A.1:

1. **Dimensiunile Sub-ferestrei:** Înălțimea sub-ferestrei: $h/h_2 = 10/5 = \mathbf{2}$. Lățimea sub-ferestrei: $w/w_2 = 10/5 = \mathbf{2}$. Fiecare sub-fereastră va avea dimensiunea de 2×2.
2. **Numărul Total de Feature-uri Extrase:** Numărul de *feature*-uri extras pentru o singură regiune este $c \times h_2 \times w_2$.

$$\text{Total Feature-uri} = 256 \times 5 \times 5 = 256 \times 25 = \mathbf{6400}$$

Exercițiul A.2 (Propagare R-CNN):

Un set de test conține 50 de imagini. Se utilizează R-CNN inițial. Pe baza informațiilor din sursă, se presupune că *selective search* generează $= 2000$ propuneri de regiuni per imagine.

Calculați numărul total de ori în care trebuie executată operația de extragere a *feature*-urilor (propagarea forward CNN) pe întregul set de test. Comparați acest număr cu numărul de rulări CNN forward necesar pentru același set de test folosind Fast R-CNN.

Rezolvare A.2:

1. **R-CNN inițial (Propagare independentă per regiune):**

$$\text{Total Propaări} = \text{Număr Imaini} \times \text{Număr Propuneri per Imaine}$$

$$\text{Total Propaări} = 50 \times 2000 = \mathbf{100,000}$$

2. **Fast R-CNN (Propagare pe întreaga imagine):**

$$\text{Total Propaări} = \text{Număr Imaini} = \mathbf{50}$$

(Diferența de la 100.000 la 50 demonstrează că principala îmbunătățire a Fast R-CNN este partajarea calculului, transformând R-CNN-ul dintr-un model lent, nefizabil în aplicațiile în timp real, într-unul mult mai eficient.)